DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Jingyao Zhang
School of Cyber Science and Engineering
Southeast University
Nanjing, China
zhangjingyao@seu.edu.cn

Abstract— Compiler-instrumented defenses (e.g., stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`) protect software against security threats, such as memory corruption, control-flow hijacking, and data leakage. However, these defenses can fail silently: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Existing compiler testing tools fail to detect such bugs. Conventional crash and differential testing only check whether a program produces correct outputs, leaving a fundamental oracle gap when a defense is silently weakened or broken.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFUZZ first extracts security invariants from compiler source code, Instruction Set Architecture (ISA) specifications, and historical bug fixes using a hybrid CoT and RAG pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. To detect violations reliably, DEFUZZ translates these invariants into programmatic bug oracles with formal validation, implementing them as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered [TBD: N] previously unknown silent failures in compiler-instrumented defenses, of which [TBD: M] have been confirmed upstream. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden software against security threats, such as memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within.

When a defense implementation goes wrong, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. Because the compiled program still runs normally on benign inputs, standard functional tests pass completely, leaving the software exposed to attack. For example, in **CVE-2023-4039** [1], GCC on AArch64 placed the saved frame pointer and return address at the bottom of the fixed stack frame, below the stack canary. When a function used `alloca` or a variable-length array, GCC allocated that dynamic buffer at a lower stack address right below the saved return address. An upward buffer overflow from the dynamic region could then overwrite the saved return address without ever touching the canary.

Existing compiler testing approaches fail to catch such silent bugs due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [2] and YARPGen [3]) check for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on benign inputs, crash and differential oracles remain blind when a defense is weakened or broken. The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can reach given target branches, but they cannot tell which branches inside millions of lines of compiler code control a defense. Even after reaching a defense branch, exposing a silent failure requires satisfying defense-specific semantic conditions, such as opaque object bounds or cross-pass length propagation, rather than generic control-flow or data-flow coverage. For instance, in a GCC bug we uncovered (PR 125079 [4]), triggering silent `_FORTIFY_SOURCE` relaxation in the `strlen` pass requires pairing `__strcat_chk(dst, src, objsize)` on an unknown-length buffer with a subsequent `strlen(dst)` call that triggers the rewrite into `__stpcpy_chk`. Random mutators rarely synthesize such coupled defense semantics. Meanwhile, building static analyzers demands heavy manual effort from experts to craft custom rules for each defense mechanism and target ISA, while unconstrained LLM agents

suffer from severe hallucinations and false bug reports.

These barriers raise a central question: how can we systematically detect silent defense failures when neither the buggy compiler nor direct LLM judgment provides a reliable oracle? To answer this question, we present **DEFUZZ**, an invariant-directed agentic framework for testing compiler-instrumented defenses. Our core insight is that although defense implementations vary across backends, their intended security guarantees follow target-independent *security invariants*. DEFUZZ translates these invariants into programmatic bug oracles implemented as ISA-specific static and dynamic checkers, closing the oracle gap. Because LLM-assisted translation can introduce errors, DEFUZZ combines formal validation in oracle construction with offline sampling audits to ensure that the checkers faithfully enforce each invariant. To trigger defense-critical logic, DEFUZZ maps each invariant to its corresponding defense-related compiler branches and guides LLM agents to synthesize inputs that satisfy both branch conditions and invariant preconditions. All bug verdicts remain strictly with deterministic checkers, avoiding both the blind spots of random mutators and LLM adjudication hallucinations.

Building DEFUZZ requires solving three main challenges. The first challenge is establishing reliable security invariants: because the compiler itself has bugs, we cannot trust it. Relying on a compiler's own behavior or implementation code as the ground truth creates circular self-validation. Our key idea is to separate intended security contracts from buggy compiler implementations by anchoring invariants in external ISA manuals, documented design assertions, and confirmed security patches, while abstracting past fixes into transferable failure patterns across defenses. The second challenge is checking these invariants with high accuracy across diverse ISAs, where distinct registers and stack layouts cause direct LLM judgment to hallucinate. DEFUZZ addresses this by translating each invariant into ISA-specific static and dynamic checkers backed by formal validation. Dual static-dynamic cross-checking and differential baseline builds further verify each target, ensuring reliable verdicts with **zero LLM involvement in bug adjudication**. The final challenge is steering test generation to both locate defense branches and satisfy their semantic preconditions without being stripped by early optimizations. DEFUZZ maps each invariant to its controlling compiler passes and branches via call-graph and AST analysis, inserting compile-time probes to trace branch execution. Guided by branch traces and invariant preconditions, LLM agents iteratively synthesize and mutate seeds to expose deep defense flaws.

We evaluated DEFUZZ on GCC and LLVM across x86-64, AArch64, and RISC-V, spanning diverse defenses such as stack canaries, CFI, shadow stacks, register scrubbing, and `_FORTIFY_SOURCE`. DEFUZZ extracted **[TBD: K]** validated security invariants and transferred historical failure patterns across defense mechanisms and backends. During fuzzing, branch-guided seed synthesis hit substantially more target defense branches than unguided baselines, while our formally validated checkers identified violations with high accuracy and avoided the false reports produced by unconstrained LLM agents. Overall, DEFUZZ discovered **[TBD: N] previously unknown silent failures in compiler-instrumented defenses**, of which **[TBD: M] have already been confirmed upstream** (Figure 1), including bugs rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

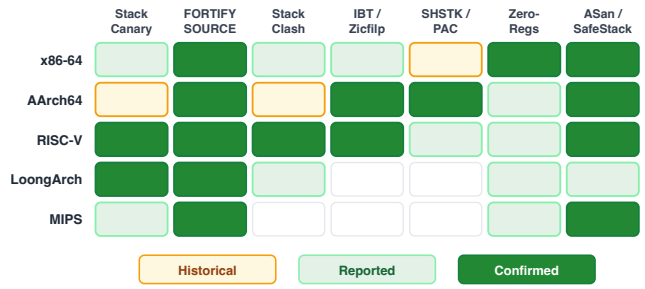


Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

In answering this central question, this paper makes the following contributions:

- **Automated Invariant Extraction:** We decouple security contracts from buggy compiler outputs by extracting security invariants from ISA specifications, design assertions, and historical bug fixes, transferring failure patterns across defenses.
- **Validated Translation into Multi-ISA Programmatic Oracles:** We translate security invariants into formally validated static and dynamic checkers across target ISAs, verifying compiled code with high accuracy and zero LLM involvement in bug adjudication.
- **Call-Graph-Guided Agentic Fuzzing:** We map invariants to defense-related compiler passes and branches, and guide specialized LLM agents with compile-time branch traces to synthesize seeds that satisfy defense triggering conditions.
- **New Silent Defense Bug Discovery:** We evaluate DEFUZZ on GCC and LLVM across multiple ISAs and discover **[TBD: N]** previously unknown silent failures in compiler-instrumented defenses, with **[TBD: M]** confirmed upstream.

II. MOTIVATION

A. Compiler-Instrumented Defenses

Compilers deploy automated defenses to protect software against security threats. Unlike standalone binary rewriters, compiler-instrumented defenses span both middle-end and backend stages of compilation. Middle-end passes analyze object bounds and control flow to insert or optimize high-level checks on the intermediate representation (IR). Backend passes enforce low-level protections by controlling stack frame

layouts, register assignments, and machine instructions across target ISAs.

Preserving security guarantees across this multi-stage pipeline is error-prone. A middle-end optimization can rewrite a defense check under flawed size assumptions, or a backend pass can misplace a stack guard slot. When such bugs occur, the compiler neither crashes nor emits warnings. Instead, it produces a working binary whose security protection is quietly weakened or broken.

B. Motivating Example

Listing 1 shows a silent defense failure in GCC's `_FORTIFY_SOURCE` implementation (PR 125079 [4]) discovered by DeFuzz. This bug had remained hidden for 14 years before upstream maintainers fixed it in the GCC mainline. Under `-D_FORTIFY_SOURCE=2`, GCC replaces unsafe string calls such as `strcat(dst, src)` on the 64-byte buffer `dst` with the bounds-checked builtin `__strcat_chk(dst, src, 64)`. At runtime, this builtin checks that the existing string in `dst` plus the appended `src` fits within 64 bytes, aborting execution on overflow.

```
1  char dst[64];
2
3  size_t append_and_measure(const char *src) {
4    init_opaque(dst); // hides prefix len k (k = 8)
5
6    // 1. Fortified call: __strcat_chk(dst, src, 64)
7    strcat(dst, src);
8
9    // 2. get_string_length() rewrites call to:
10   // __stpcpy_chk(dst + k, src, 64)
11   // BUG: bound stays 64 instead of 64 - k!
12   return strlen(dst);
13 }
```

Listing 1. Silent `_FORTIFY_SOURCE` relaxation in GCC (PR 125079).

In Listing 1, `init_opaque(dst)` initializes the 64-byte buffer `dst` with an 8-byte string ($k = 8$) whose length is unknown at compile time, followed by `strcat(dst, src)` and `strlen(dst)`. Executing `strcat(dst, src)` and `strlen(dst)` naively would scan `dst` twice: once to find the end of the prefix, and again to measure the combined string. To avoid this redundant scan, GCC's `tree-ssa-strlen` pass invokes `get_string_length()` in `tree-ssa-strlen.cc` when it processes the trailing `strlen(dst)` call. Inside `get_string_length()`, the `BUILT_IN_STRCAT_CHK` branch rewrites the earlier `__strcat_chk(dst, src, 64)` into `__stpcpy_chk(dst + k, src, 64)`, which copies `src` to `dst + k` and returns the end pointer to compute `strlen(dst)` directly. However, while GCC advances the destination pointer by k bytes, it fails to subtract k from the size bound, passing 64 instead of $64 - k = 56$. As a result, the runtime check still permits writing 64 bytes starting from `dst + 8`, allowing an 8-byte buffer overflow without triggering an abort.

C. Limitations of Existing Approaches

Existing compiler testing tools fail to detect this vulnerability due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers such as Csmith [2], YARPGen [3], and EMI [5] only check whether the compiler crashes or whether compiled binaries produce consistent outputs on benign inputs. In Listing 1, GCC compiles the code cleanly, and any benign string `src` produces the exact same return value across all optimization levels. Because crash and differential testing do not check whether the binary enforces the reduced bound $(64 - k)$, they remain blind when the defense is weakened.

The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can steer execution toward given target branches, but they cannot tell which branches inside millions of lines of compiler source code control a defense, such as the `BUILT_IN_STRCAT_CHK` branch in `get_string_length()`. Even after reaching that branch, triggering the bug in Listing 1 requires three coupled conditions on `dst`: the buffer must have a known allocation size so GCC emits `__strcat_chk(dst, src, 64)`, its prefix length k must remain unknown at the append site so folding is deferred, and a subsequent `strlen(dst)` call must query the length to trigger the rewrite into `__stpcpy_chk(dst + k, src, 64)`. Random mutators rarely chain an opaque initialization, a fortified append, and a subsequent `strlen` call on the same buffer.

Beyond fuzzing, static analysis requires heavy manual effort from human experts to craft custom rules for each defense mechanism and target ISA. Meanwhile, using LLMs to audit compiler source code end-to-end causes severe hallucinations and false bug reports. To overcome these limitations, a testing framework must locate defense-related compiler branches and guide seed generation toward them. The generated seeds must satisfy the branch conditions so the compiler emits the defense code in the binary. The framework must then run the compiled binary with defense-triggering inputs and use deterministic oracles to check whether the protection is silently weakened or broken.

III. Design of DeFuzz

In this section, we present the design of DeFuzz, an invariant-directed framework for discovering silent failures in compiler-instrumented defenses. Figure 2 illustrates its overall workflow, which consists of four core steps: invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract security invariants from compiler source code, ISA specifications, and historical defects using a hybrid CoT and RAG pipeline. With these invariants established, we map them to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Next, we translate these invariants into programmatic bug oracles with formal validation. We implement these oracles as static and dynamic checkers across target ISAs to verify

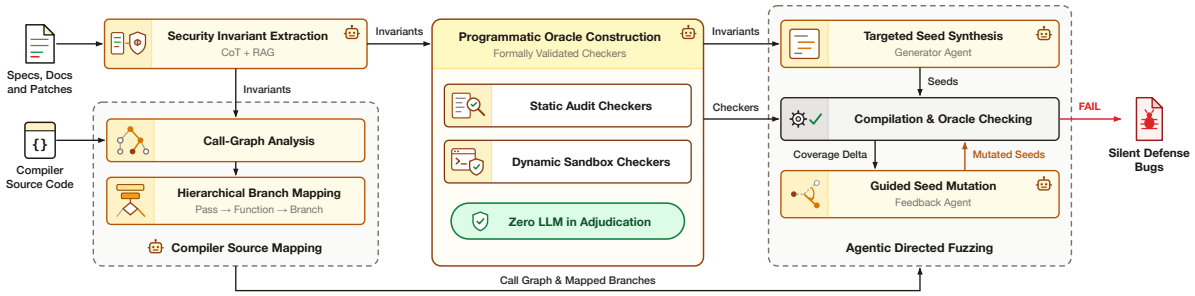


Fig. 2. Overview of DEFUZZ. The framework operates across four core steps: (1) extracting security invariants from compiler source code, ISA specifications, and historical defects via a hybrid CoT and RAG pipeline; (2) mapping invariants to compiler passes and decision branches; (3) translating invariants into formally validated static and dynamic oracles across target ISAs; and (4) performing agentic directed fuzzing, where a generator agent synthesizes probe seeds and a feedback agent steers iterative exploration under programmatic oracle verification.

whether compiled binaries preserve intended defenses, avoiding false reports from model hallucination. Finally, our agentic directed fuzzing coordinates multiple specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations, with our programmatic oracles evaluating and verifying each candidate. We discuss each step in detail in the following subsections.

A. Security Invariant Generation

The first step extracts a catalog of verifiable security invariants from compiler source code, architecture manuals, and historical bug patches.

1) Formal Invariant Contract: A *security invariant* is a rule that a compiler-instrumented defense must satisfy under its threat model to protect a program. When compiled code violates this rule, the defense is silently weakened or broken, even if the binary compiles without warnings and passes functional tests.

To support automated checking across backends, DeFuzz represents each security invariant ϕ as a six-part tuple:

$$\phi = \langle \mathcal{M}, \mathcal{A}, \mathcal{P}_{\text{cond}}, \mathcal{S}_{\text{inv}}, \mathcal{O}_{\text{phenom}}, \mathcal{E}_{\text{prov}} \rangle \quad (1)$$

Here, $\mathcal{M}$ is the target defense mechanism (e.g., stack canary, CFI, or `_FORTIFY_SOURCE`), $\mathcal{A}$ lists the applicable ISAs, $\mathcal{P}_{\text{cond}}$ defines the source-level activation preconditions (e.g., dynamic stack allocation via VLA or `alloca`), $\mathcal{S}_{\text{inv}}$ states the invariant rule, $\mathcal{O}_{\text{phenom}}$ describes the observable violation signal, and $\mathcal{E}_{\text{prov}}$ stores the supporting source evidence.

DeFuzz separates the abstract violation signal $\mathcal{O}_{\text{phenom}}$ from the concrete checking code. For instance, "the stack canary does not sit between local buffers and the saved return address" is an abstract violation signal. Checking stack offsets in DWARF debug records or assembly output is a concrete detection recipe implemented later by the oracle (§III-C).

2) Candidate Extraction via Segmented CoT: To avoid circular self-validation from buggy compilers, DeFuzz mines invariants from three trusted sources: compiler source comments and assertions, official ISA and ABI manuals, and historical CVE reports with upstream fix patches.

In production compilers, the implementation of a single defense is spread across target-independent middle-end passes and target-specific backend files. Feeding millions of lines of compiler source code into an LLM at once causes severe context loss and hallucination. Therefore, DeFuzz splits the compiler source tree into focused partitions along pass and function boundaries. It then uses a segmented Chain-of-Thought (CoT) prompt [6] to analyze each partition independently. For each code block, the model identifies the protected target, the activation condition, and the intended security guarantee. Every candidate invariant must cite exact source files and line numbers as provenance evidence $\mathcal{E}_{\text{prov}}$.

3) Cross-Mechanism Transfer via Hybrid RAG: Segmented CoT extracts rules within individual defenses, but different defenses often share the same underlying failure modes. To find similar flaws across defenses and backends, DeFuzz uses a Retrieval-Augmented Generation (RAG) [7] pipeline based on abstract failure transfer.

Standard code search matches specific variable or function names, which traps the search inside a single defense mechanism. Instead, DeFuzz generalizes confirmed historical bugs into mechanism-independent failure patterns through four steps:

1) **Distillation:** Summarize a confirmed bug into a general failure pattern. For example, CVE-2023-4039 is generalized from "AArch64 VLA overflows return address before canary" into "dynamic stack allocations bypass statically calculated guard boundaries."
2) **Analogy Filtering:** Scan candidate code blocks across other defenses (such as SafeStack, ShadowCallStack, or PAC) and discard superficial keyword matches that do not share the same structural logic.
3) **Specialization:** Map the general failure pattern onto the target defense's data structures, such as shadow stack pointers or PAC signing instructions.
4) **Entailment Check:** Verify that the specialized invariant rule logically follows from the retrieved source code and comments.

a) Dual-Aspect Query Formulation: To search the compiler codebase effectively, DeFuzz encodes each failure

TABLE I
TRANSFERRED INVARIANTS BY RETRIEVAL BACKEND (24 PROBES, 4,496 GCC-16.1 CHUNKS).

Category	Count	Note
Intersection (BM25 $\cap$ embed)	5	Same seed & chunk, isomorphic statement
BM25-only	4	Embedding missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	**11**	$9+7-5$; all verified as novel

pattern into two query forms: a dense semantic summary (`root_cause_phrase`) and sparse code tokens (`critical_tokens`, such as register macros and pass flags).

We tested sparse BM25 and dense embedding retrieval using 24 historical bug probes across 4,496 GCC-16.1 source chunks. BM25 captures exact identifier matches (e.g., `ix86_zero_call_used_regs`, `SUBREG_PROMOTED`), while dense retrieval finds similar defense logic written with different variable names across backends (e.g., ARM CMSE register clearing and RISC-V saturating truncations). After deduplicating by seed, code chunk, and invariant statement, the two retrievers find 5 shared invariants, 4 BM25-only invariants, and 2 embedding-only invariants, yielding 11 novel invariants in total (Table I).

4) Grounding Gates and Catalog Maintenance: Before entering the final catalog Φ, every candidate invariant from CoT or RAG must pass three filtering gates:

- **Gate 1 (Evidence Check):** Confirms that the cited source files, AST structures, or architecture manuals directly support the stated precondition and security rule.
- **Gate 2 (Observability Check):** Verifies that the violation signal $\mathcal{O}_{\text{phenom}}$ can be observed deterministically in compiled binaries or runtime traces.
- **Gate 3 (Deduplication):** Computes cosine similarity against existing invariants in Φ and drops candidates above an 85.0% threshold to avoid duplicate testing.

Validated invariants are stored in the catalog $\Phi = \{\phi_1, \phi_2, \ldots, \phi_n\}$. Whenever online fuzzing discovers a new silent defense bug, DeFuzz distills its root cause and feeds it back into the RAG probe pool to uncover related bugs in future runs.

B. Compiler Source Mapping

With the security invariants established, DeFuzz maps each invariant to the compiler passes and decision branches that control its implementation.

1) MCP-Guided Codebase Navigation: Production compilers such as GCC and LLVM contain millions of lines of C and C++ code. Finding the exact files and conditional branches that govern a security defense is difficult for automated tools. To search this codebase accurately, DeFuzz equips a coding agent with read-only code analysis tools via the Model Context Protocol (MCP).

The MCP toolkit provides four analysis tools: `call-graph` to trace function calls across optimization passes, `ast-parser` to inspect C/C++ syntax trees, `alia-ast` to examine intermediate representations, and `ripgrep` for fast symbol and macro lookup. Using these tools, the agent traces how each defense is lowered from middle-end passes into backend machine instructions.

2) Three-Level Decision Hierarchy: Through MCP-guided code navigation, the agent maps each security invariant ϕ to a three-level hierarchy in the compiler source tree:

- **Level 1 (Compiler Pass):** Locates the middle-end or backend pass that implements the defense (e.g., `pass_stack_protect` in GCC or `StackProtectorPass` in LLVM).
- **Level 2 (Target Functions):** Pinpoints the functions that calculate stack layouts, insert guard checks, or lower instructions (e.g., `expand_used_vars()` in GCC or `LayoutStackNodes()` in LLVM).
- **Level 3 (Critical Structs and Branches):** Identifies the exact data structures and conditional branches that control defense placement, such as `has_vla`, `stack_protect_decl`, and `canary_loc`.

This three-level mapping links abstract invariant preconditions directly to concrete decision branches in the compiler source code.

3) Compile-Time Branch Probing (`GCC_INST`): Once the Level-3 decision branches are mapped, the coding agent instruments the compiler source code by inserting lightweight logging probes:

```
DEFUZZ_LOG("SP#a3f2:hit", slot);
```

Here, `"SP#a3f2"` is a unique identifier (UID) that encodes the defense mechanism, source file, function, and branch index. These probes run during compilation passes, recording which defense branches were taken and logging internal values such as stack slot offsets. Because the probes only record compiler-internal states at build time, they never alter the generated machine code or add runtime overhead to the emitted target binary.

After inserting the probes, DeFuzz rebuilds the compiler into an instrumented driver, denoted `GCC_INST` (or `CLANG_INST` for LLVM). Whenever `GCC_INST` compiles a test seed, it writes a deterministic UID trace to disk, showing the fuzzing loop whether targeted defense branches were exercised.

C. Programmatic Oracle Construction

Next, DEFUZZ translates the extracted security invariants into programmatic bug oracles across target ISAs with formal validation, achieving high diagnostic accuracy without model hallucination (Figure 3).

1) Oracle Contract and Four-State Verdict Matrix: DeFuzz defines each bug oracle as a deterministic contract. Let $\mathcal{P}$ denote the space of compiled program binaries, $\mathcal{I} =$

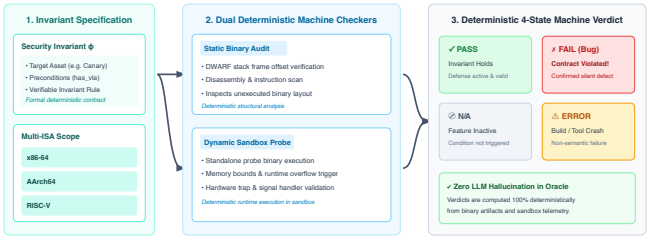


Fig. 3. Multi-ISA deterministic oracle architecture. Invariant contracts are specialized across target ISAs and instantiated into dual static audit and dynamic sandbox checkers, yielding a deterministic four-state verdict without LLM adjudication.

$\{\text{x86-64}, \text{AArch64}, \text{RISC-V}\}$ be the set of target ISAs, and $\mathcal{S} \subseteq \Phi$ be the set of security invariants. The oracle operates as a deterministic function:

$$\mathcal{O} : \mathcal{P} \times \mathcal{I} \times \mathcal{S} \longrightarrow \{\text{PASS}, \text{FAIL}, \text{NA}, \text{ERROR}\} \quad (2)$$

Each output state carries a clear meaning:

- **PASS:** The compiled binary satisfies the security invariant on the target architecture.
- **FAIL:** The binary violates the security invariant, confirming a silent compiler defense bug.
- **NA (Not Applicable):** The target defense was disabled, or the test seed did not trigger the invariant's precondition.
- **ERROR:** Compilation failed due to syntax errors or environment faults. The system logs this state as a build diagnostic and never reports it as a defense bug.

LLM agents never adjudicate or override these verdicts. Every bug report requires deterministic verification by the oracle function $\mathcal{O}$, ensuring zero LLM involvement in bug adjudication.

2) Dual Static and Dynamic Checkers: Compiler defense failures manifest in two distinct forms. Some failures leave structural defects in emitted binary or assembly code, which can be observed without executing the program. Other failures only emerge during runtime execution, where compiler optimizations silently bypass protection boundaries. To catch both classes of bugs deterministically, DeFuzz builds two complementary types of programmatic checkers. Static checkers inspect unexecuted compiler artifacts, while dynamic checkers monitor runtime execution traces. All bug verdicts are computed directly by these checkers, ensuring zero LLM involvement in bug adjudication.

a) Static Checkers (Binary and Assembly Audit): Static checkers inspect unexecuted compiler outputs, including assembly listings, machine instructions, stack frame layouts, and ELF symbol tables. They verify structural defense properties without executing the test binary.

Consider stack canary protection in GCC as an example. When checking a canary at function return, the compiler loads the reference canary into a scratch register and compares it against the stack guard slot. In vulnerable compiler versions, the compiler forgot to clear this scratch register before returning to the caller. This flaw allowed the secret canary value to leak through caller-accessible registers, enabling attackers to read the canary and bypass stack protection entirely.

```
// Scan registers right after return
void audit_canary_leak(uintptr_t canary) {
  uintptr_t r0, r1, r2 /* ... */;

  asm volatile(
    "mov %%rax, %0; mov %%rbx, %1; mov %%rcx, %2"
    // ... (for all target ISA registers)
    : "=r"(r0), "=r"(r1), "=r"(r2)
  );

  // FAIL if any register retains canary
  if (r0 == canary || r1 == canary /* ... */)
    emit_verdict(FAIL, "Canary leaked");
  else
    emit_verdict(PASS, "Canary cleared");
}
```

Listing 2. Static/inline checker for canary register leakage.

Listing 2 shows how DeFuzz detects this flaw. The checker audits the emitted assembly and executes an inline probe right after the epilogue, scanning all general-purpose registers for the canary secret. If any register still holds the canary value, the checker records a FAIL verdict. Otherwise, it records a PASS.

b) Dynamic Checkers (Runtime Sandbox Verification): Dynamic checkers execute compiled test binaries inside an isolated sandbox with dedicated signal handlers and memory fault traps. They evaluate runtime security guarantees that static layout analysis cannot reliably verify.

Recall our motivating example in GCC (PR 125079 [4], §II-B), where `tree-ssa-strlen` rewrites `__strcat_chk(dst, src, 64)` into `__stpcpy_chk(dst + k, src, 64)` without subtracting the prefix length k from the remaining bound. Listing 3 shows the dynamic verification harness for this invariant. After initializing a 64-byte buffer with an 8-byte prefix via an opaque helper, the test appends a 60-byte string via `__strcat_chk(buf, src60, 64)` followed by `strlen(buf)`. Because $8 + 60$ exceeds the 64-byte buffer, a working defense must abort with `SIGABRT`, which the harness catches to record PASS. Because GCC leaves the bound at 64 instead of $64 - 8 = 56$, the 60-byte write finishes without aborting and reaches the failure sentinel, flagging a deterministic FAIL.

```c
void test_fortify_strcat(const char *src60) {
  char buf[64];
  init_opaque_8bytes(buf); // opaque strlen == 8
  setup_sigabrt_handler(); // trap fortify abort

  // Rewritten to __stpcpy_chk(buf + 8, src60, 64)
  // Missing subtraction allows 8-byte overflow
  __builtin___strcat_chk(buf, src60, sizeof(buf));
  sink = __builtin_strlen(buf); // triggers fold

  emit_verdict(FAIL, "Overflow unchecked");
}

void sigabrt_handler(int sig) {
  emit_verdict(PASS, "Overflow trapped");
  _exit(0);
}
```

Listing 3. Dynamic checker for `_FORTIFY_SOURCE` bounds.

c) Redundant Cross-Validation: When an invariant can be observed both statically and dynamically, DeFuzz generates dual checkers to cross-validate results. For example, the static checker inspects the compiler IR and assembly to verify that `__stpcpy_chk` receives a decremented size argument (`objsize - strlen(dst)`). Meanwhile, the dynamic checker confirms that an overflowing append triggers a runtime abort. This dual design prevents false negatives where the compiler emits defense calls with stale or permissive bounds. Every reported bug represents a deterministic, reproducible compiler defense failure without human or LLM bias.

3) Target-Specific Specialization Across ISAs: A security invariant defines a universal defense rule, but its machine-level enforcement depends on target architecture semantics. Different ISAs use distinct instruction sets, register conventions, and stack frame layouts. Therefore, for each invariant $s \in \mathcal{S}$ and target ISA $i \in \mathcal{I}$, DeFuzz generates a dedicated checker $C_{s,i}$.

Consider stack canary protection across architectures. On x86-64, the `call` instruction pushes the return address directly onto the stack above the saved frame pointer. On AArch64 and RISC-V, the CPU holds the return address in a link register (`x30` or `ra`), spilling it to the stack only in non-leaf functions. Thus, an x86-64 layout checker verifies buffer offsets relative to the frame pointer. In contrast, an AArch64 checker tracks whether dynamic allocations place buffers above the saved link register slot. Similarly, register leak checkers (Listing 2) scan `rax`–`r15` on x86-64, `x0`–`x30` on AArch64, and `x1`–`x31` on RISC-V. This target-specific specialization prevents valid architectural differences from triggering false bug reports.

4) Formal Validation and Sampling Audit: Because translating invariants into target-specific checker code involves LLM assistance, an unverified checker could misstate stack offset signs, bound arithmetic, or register sets. To prevent translation errors from corrupting bug verdicts, DeFuzz validates generated checkers before deployment through formal validation alongside offline manual sampling audits (**[TBD: detail concrete Step 3 formal validation mechanism once finalized]**). Only checkers that pass validation enter the online fuzzing loop.

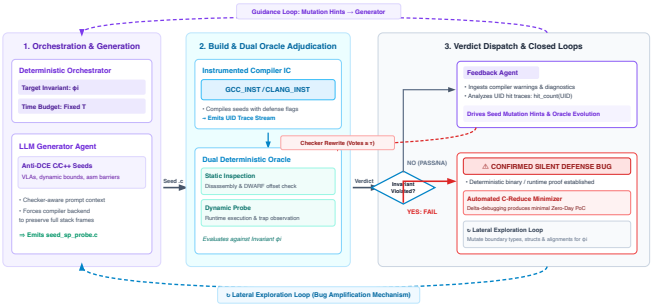


Fig. 4. Online closed-loop agentic fuzzing workflow. A deterministic orchestrator schedules invariant targets under fixed budgets. A generator agent synthesizes anti-DCE seeds, which are built by `GCC_INST` and evaluated by the oracle. Failures trigger C-Reduce minimization and lateral amplification, while non-violations feed telemetry into guidance and checker evolution loops.

5) Differential Configuration Profiling: To isolate defense-specific behaviors, each checker runs the same test seed under two compilation profiles: a defense-enabled build (e.g., `-fstack-protector-strong`) and an unprotected baseline build. Comparing both outputs confirms that observed layout changes or runtime traps come from the target defense rather than unrelated optimization passes. When a checker confirms a FAIL verdict under the defense-enabled build, the oracle emits a bug alert with full binary and execution trace evidence.

D. Agentic Directed Fuzzing

Finally, with the invariant catalog, mapped compiler branches, and programmatic oracles in place, DeFuzz runs its online directed fuzzing loop (Figure 4). Guided by the mapped compiler locations, specialized agents synthesize and mutate program seeds to hit targeted defense branches, while programmatic oracles evaluate every candidate binary.

1) Deterministic Orchestration Pipeline: Rather than letting LLM agents run freely, DeFuzz controls the fuzzing workflow with a deterministic orchestrator. At campaign startup, the orchestrator loads target invariants from the catalog Φ into a queue Q. It locks one invariant ϕ_i at a time along with its ISA-specific checkers, assigning each target an equal time and execution budget T.

Every fuzzing iteration follows a fixed sequence: seed synthesis $\rightarrow$ compilation with `GCC_INST` $\rightarrow$ UID branch trace collection $\rightarrow$ oracle evaluation $\rightarrow$ feedback dispatch. All intermediate states (including generated seeds, compiler logs, and checker verdicts) are recorded on a versioned blackboard $\mathcal{B}$. Pinning a blackboard version deterministically reproduces every test input, compiler trace, and oracle result. Algorithm 1 summarizes this control flow.

2) Checker-Aware Seed Synthesis and DCE Evasion: Random program generators rarely trigger deep compiler defense bugs because their generated code is often removed by early optimizations. In particular, Dead Code Elimination (DCE) strips away unused buffer allocations or redundant pointer writes before backend defense passes run.

Algorithm 1 Online Closed-Loop Agentic Fuzzing Pipeline

Require: Invariant queue Q, Budget T, Instrumented driver `GCC_INST`, Oracle suite $\mathcal{O}$

Initialize versioned blackboard $\mathcal{B}$, Historical seed pool $\mathcal{H} \leftarrow \emptyset$
for target invariant $\phi_i \in Q$ **do**
  Set round budget $t \leftarrow T$
  while $t > 0$ **do**
    $\mathcal{B}$.seed $\leftarrow$ GeneratorAgent($\phi_i, \mathcal{B}$.feedback)
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{\mathcal{B}.\text{seed}\}$
    b, trace $\leftarrow$ Compile(`GCC_INST`, $\mathcal{B}$.seed, ϕ_i.flags)
    $v \leftarrow \mathcal{O}(b, \phi_i.\text{ISA}, \phi_i)$
    if $v ==$ FAIL **then**
      $\mathcal{B}$.poc $\leftarrow$ MinimizerAgent($\mathcal{B}$.seed, $\mathcal{O}, \phi_i$)
      yield ConfirmedBugFound($\mathcal{B}$.poc, ϕ_i)
      LateralDivergence($\phi_i, \mathcal{B}$.seed)
      break
    else if $v \in \{\text{PASS}, \text{NA}, \text{ERROR}\}$ **then**
      $\mathcal{B}$.feedback $\leftarrow$ FeedbackAgent(v, trace, $\mathcal{B}$.seed)
      if $\mathcal{B}$.feedback.rewrite_votes $\geq \tau$ **then**
        ϕ_i.checker $\leftarrow$ TriggerCheckerRewrite(ϕ_i.checker)
        ReplaySeedPool($\mathcal{H}, \phi_i$.checker, `GCC_INST`)
      end if
    end if
    $t \leftarrow t - 1$
  end while
end for

To survive optimization and hit targeted Level-3 defense branches, our Generator Agent synthesizes C/C++ probe seeds conditioned on the invariant ϕ_i, the target ISA, and the mapped compiler conditions (§III-B). The agent embeds three anti-DCE patterns into generated seeds:

- Variable-length arrays (VLAs) whose sizes depend on runtime inputs such as `argc`.
- Escaping buffer pointers marked with `volatile` qualifiers or `asm volatile` memory barriers.
- External or non-inlinable function calls that force the compiler to preserve full stack frames and defense checks.

3) Adaptive Telemetry Feedback and Checker Evolution: When the oracle returns a non-bug verdict (PASS, NA, or ERROR), the Feedback Agent inspects the compiler diagnostics, UID branch hit counts, and checker traces from `GCC_INST`. It then drives two closed-loop adaptation mechanisms:

- **Mutation Guidance Loop:** If the UID trace shows that the seed missed a required compiler branch (for example, `has_vla` evaluated to false), the Feedback Agent produces concrete mutation hints telling the Generator Agent which control-flow or variable types to adjust in the next round.
- **Checker Evolution Loop:** If multiple rounds reveal an incomplete checker rule (reaching a vote threshold τ), DeFuzz updates the ISA checker and immediately replays all historical seeds in $\mathcal{H}$ to catch any previously missed violations.

4) Automated Bug Minimization and Lateral Amplification: When a checker outputs FAIL, the oracle confirms a silent defense bug based solely on deterministic machine evidence. DeFuzz then passes the failing seed to a Minimizer Agent, which pairs C-Reduce with oracle verification. Every reduced candidate is recompiled with `GCC_INST` and rechecked by $\mathcal{O}$ before acceptance. This step strips away unrelated code while preserving the exact memory layout or optimization pattern that triggers the bug, producing a minimal Proof-of-Concept (PoC).

a) Lateral Bug Amplification: Finding a bug does not stop testing for that invariant. Instead, the orchestrator starts a lateral amplification loop, instructing the Generator Agent to mutate surrounding syntax (such as data types, struct nesting, and stack alignment attributes) around the failing PoC. This targeted exploration checks whether the same root cause breaks sibling optimization passes or other backend architectures.

IV. Implementation

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build, and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

References

[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[4] GCC Bugzilla, "Bug 125079: tree-ssa-strlen get_string_length: __strcat_chk to __stpcpy_chk drops bound subtraction," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079, 2026.

[5] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.